AI Assistant Coding

Assignment 12.3-1

Name : SUNKARI SAI 2303A52492 **Batch:** 50

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).

2. Implement the following sorting algorithms using AI assistance:

o Quick Sort

o Merge Sort

3. Measure and compare runtime performance for large datasets.

4. Write a function to display the top 10 students based on CGPA.

Expected Outcome

- **Correctly sorted student records.**
- **Performance comparison between Quick Sort and Merge Sort.**
- **Clear output of top-performing students.**

#PROMPT

"""generate a program that stores student records (Name, Roll Number, CGPA).

Implement the following sorting algorithms:

o Quick Sort

o Merge Sort

Measure and compare runtime performance for large datasets,

and Write a function to display the top 10 students based on CGPA"""

#CODE

```
import random
import time
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa
def quick_sort(students):
    if len(students) <= 1:
        return students
    pivot = students[len(students) // 2].cgpa
    left = [x for x in students if x.cgpa > pivot]
    middle = [x for x in students if x.cgpa == pivot]
    right = [x for x in students if x.cgpa < pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(students):
    if len(students) <= 1:
        return students
    mid = len(students) // 2
    left = merge_sort(students[:mid])
    right = merge_sort(students[mid:])
    return merge(left, right)
def merge(left, right):
```

```
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa > right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def display_top_students(students, n=10):
    top_students = students[:n]
    for student in top_students:
        print(f"Name: {student.name}, Roll Number: {student.roll_number}, CGPA:
{student.cgpa}")
def generate_random_students(num_students):
    students = []
    for i in range(num_students):
        name = f"Student_{i+1}"
        roll_number = i + 1
        cgpa = round(random.uniform(0, 10), 2)
        students.append(Student(name, roll_number, cgpa))
    return students
def main():
    num_students = 10000
```

```
    students = generate_random_students(num_students)

    start_time = time.time()
    sorted_students_quick = quick_sort(students)
    end_time = time.time()
    print(f"Quick Sort Time: {end_time - start_time:.4f} seconds")

    start_time = time.time()
    sorted_students_merge = merge_sort(students)
    end_time = time.time()
    print(f"Merge Sort Time: {end_time - start_time:.4f} seconds")

    print("\nTop 10 Students based on CGPA:")
    display_top_students(sorted_students_quick)
if __name__ == "__main__":    main()
```

#OUTPUT

```
Quick Sort Time: 0.0253 seconds
Merge Sort Time: 0.0539 seconds

Top 10 Students based on CGPA:
Name: Student_3400, Roll Number: 3400, CGPA: 10.0
Name: Student_7083, Roll Number: 7083, CGPA: 10.0
Name: Student_7559, Roll Number: 7559, CGPA: 10.0
Name: Student_9915, Roll Number: 9915, CGPA: 10.0
Name: Student_1047, Roll Number: 1047, CGPA: 9.99
Name: Student_1440, Roll Number: 1440, CGPA: 9.99
Name: Student_1611, Roll Number: 1611, CGPA: 9.99
```

Task 2: Implementing Bubble Sort with AI Comments

- **Task: Write a Python implementation of Bubble Sort.**
- **Instructions:**

- **Students implement Bubble Sort normally.**
- **Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).**
- **Request AI to provide time complexity analysis.**
- **Expected Output:**
- **A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.**

#PROMPT

"""Write a Python implementation of Bubble Sort.explaining key logic (like swapping, passes, and termination).provide time complexity analysis"""

#CODE

```
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n - i - 1):
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  # Swapping
                swapped = True
        # If no swapping occurred, the array is already sorted
        if not swapped:
            break
# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
bubble_sort(arr)
```

print("Sorted array is:", arr)

#OUTPUT

```
Sorted array is: [11, 12, 22, 25, 34, 64, 90]
```

Task 3: Quick Sort and Merge Sort Comparison

- **Task: Implement Quick Sort and Merge Sort using recursion.**
- **Instructions:**
- **Provide AI with partially completed functions for recursion.**
- **Ask AI to complete the missing logic and add docstrings.**
- **Compare both algorithms on random, sorted, and reverse-sorted lists.**
- **Expected Output:**
- **Working Quick Sort and Merge Sort implementations.**
- **AI-generated explanation of average, best, and worst-case complexities.**

#PROMPT

""" Implement Quick Sort and Merge Sort using recursion and add docstrings. Compare both algorithms on random,

sorted, and reverse-sorted lists.generate explanation of average, best, and worst-case complexities."""

#CODE

```
def quick_sort(arr):
    """

    Quick Sort is a divide-and-conquer algorithm that sorts an array by selecting a 'pivot' element and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively.

    Time Complexity:

    - Average Case: O(n log n)

    - Best Case: O(n log n) (when the pivot divides the array into two equal halves)

    - Worst Case: O(n^2) (when the smallest or largest element is always chosen as the pivot)
```

```
    Args:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new sorted list.
    """
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    """Merge Sort is a divide-and-conquer algorithm that sorts an array by recursively dividing
it into two halves, sorting each half, and then merging the sorted halves back together.
    Time Complexity:
    - Average Case: O(n log n)
    - Best Case: O(n log n)
    - Worst Case: O(n log n)
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    """Merge function that takes two sorted lists and merges them into a single sorted list."""
```

```
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
import random
def generate_random_list(size):
    """Generates a list of random integers of a given size."""
    return [random.randint(0, 100) for _ in range(size)]
def main():
    # Test on random list
    random_list = generate_random_list(1000)
    print("Testing on random list:")
    print("Quick Sort Time:", timeit.timeit(lambda: quick_sort(random_list), number=1))
    print("Merge Sort Time:", timeit.timeit(lambda: merge_sort(random_list), number=1))

    # Test on sorted list
    sorted_list = sorted(random_list)
    print("\nTesting on sorted list:")
    print("Quick Sort Time:", timeit.timeit(lambda: quick_sort(sorted_list), number=1))
    print("Merge Sort Time:", timeit.timeit(lambda: merge_sort(sorted_list), number=1))
```

```
    # Test on reverse-sorted list
    reverse_sorted_list = sorted(random_list, reverse=True)
    print("\nTesting on reverse-sorted list:")
    print("Quick Sort Time:", timeit.timeit(lambda: quick_sort(reverse_sorted_list), number=1))
    print("Merge Sort Time:", timeit.timeit(lambda: merge_sort(reverse_sorted_list), number=1))
if __name__ == "__main__":
    import timeit
    main()
```

#OUTPUT

```
Testing on random list:
Quick Sort Time: 0.001412599998730002
Merge Sort Time: 0.0035110999997414183

Testing on sorted list:
Quick Sort Time: 0.0009443000017199665
Merge Sort Time: 0.002263499998662155

Testing on reverse-sorted list:
Quick Sort Time: 0.0008041000000957865
Merge Sort Time: 0.002231400001619477
```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products,

each with attributes like product ID, name, price, and stock quantity. Store staff

need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

Task:

• Use AI to suggest the most efficient search and sort algorithms for this

use case.

• Implement the recommended algorithms in Python.

• Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

• A table mapping operation → recommended algorithm → justification.

• Working Python functions for searching and sorting the inventory.

#PROMPT

"""An inventory management system in a retail store contains thousands of products, each with attributes such as

product ID, name, price, and stock quantity. Store staff need to quickly search for products by ID or name and

sort products by price or stock quantity for stock analysis. Suggest the most efficient searching and sorting

algorithms for this scenario, considering the dataset size, frequent updates, and performance needs.

Provide a table mapping each operation to the recommended algorithm with a short justification.

Then implement the recommended algorithms in Python, including functions to search products by ID and name and to

sort the inventory by price and stock quantity. Include example data and briefly mention the time complexity of the

algorithms used."""

#CODE

```
def main():
    # Example data
    inventory = [
        {"product_id": 1, "name": "Laptop", "price": 999.99, "stock_quantity": 10},
        {"product_id": 2, "name": "Smartphone", "price": 499.99, "stock_quantity": 50},
        {"product_id": 3, "name": "Headphones", "price": 199.99, "stock_quantity": 100},
        {"product_id": 4, "name": "Monitor", "price": 299.99, "stock_quantity": 20},
        {"product_id": 5, "name": "Keyboard", "price": 49.99, "stock_quantity": 200},
    ]


    # Search by product ID
```

```
    product = search_by_id(inventory, 3)
    print("Search by ID (3):", product)

    # Search by product name
    product = search_by_name(inventory, "Monitor")
    print("Search by Name (Monitor):", product)

    # Sort by price
    sorted_by_price = sort_by_price(inventory)
    print("Sorted by Price:", sorted_by_price)

    # Sort by stock quantity
    sorted_by_stock = sort_by_stock(inventory)
    print("Sorted by Stock Quantity:", sorted_by_stock)
def search_by_id(inventory, product_id):
    """Search for a product by its ID using binary search.
    Time Complexity: O(log n)"""
    inventory.sort(key=lambda x: x["product_id"])  # Ensure inventory is sorted by product_id
    left, right = 0, len(inventory) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if inventory[mid]["product_id"] == product_id:
            return inventory[mid]
        elif inventory[mid]["product_id"] < product_id:
            left = mid + 1
        else:
            right = mid - 1
    return None
def search_by_name(inventory, name):
```

```
    """Search for a product by its name using linear search.
    Time Complexity: O(n)"""
    for product in inventory:
        if product["name"].lower() == name.lower():
            return product
    return None
def sort_by_price(inventory):
    """Sort products by price using Quick Sort.
    Time Complexity: O(n log n) on average."""
    if len(inventory) <= 1:
        return inventory
    pivot = inventory[len(inventory) // 2]["price"]
    left = [x for x in inventory if x["price"] < pivot]
    middle = [x for x in inventory if x["price"] == pivot]
    right = [x for x in inventory if x["price"] > pivot]
    return sort_by_price(left) + middle + sort_by_price(right)
def sort_by_stock(inventory):
    """Sort products by stock quantity using Merge Sort.
    Time Complexity: O(n log n)"""
    if len(inventory) <= 1:
        return inventory
    mid = len(inventory) // 2
    left = sort_by_stock(inventory[:mid])
    right = sort_by_stock(inventory[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
```

```
        if left[i]["stock_quantity"] < right[j]["stock_quantity"]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
if __name__ == "__main__":
    main()
```

#OUTPUT

```
Search by ID (3): {'product_id': 3, 'name': 'Headphones', 'price': 199.99, 'stock_quantity': 100}
Search by Name (Monitor): {'product_id': 4, 'name': 'Monitor', 'price': 299.99, 'stock_quantity': 20}
Sorted by Price: [{'product_id': 5, 'name': 'Keyboard', 'price': 49.99, 'stock_quantity': 200}, {'product_id': 3, 'name': 'Headphones', 'price': 199.99, 'stock_quantity': 100}, {'product_id': 4, 'name'
: 'Monitor', 'price': 299.99, 'stock_quantity': 20}, {'product_id': 2, 'name': 'Smartphone', 'price': 499.99, 'stock_quantity': 50}, {'product_id': 1, 'name': 'Laptop', 'price': 999.99, 'stock_quantity
': 10}]
Sorted by Stock Quantity: [{'product_id': 1, 'name': 'Laptop', 'price': 999.99, 'stock_quantity': 10}, {'product_id': 4, 'name': 'Monitor', 'price': 299.99, 'stock_quantity': 20}, {'product_id': 2, 'na
me': 'Smartphone', 'price': 499.99, 'stock_quantity': 50}, {'product_id': 3, 'name': 'Headphones', 'price': 199.99, 'stock_quantity': 100}, {'product_id': 5, 'name': 'Keyboard', 'price': 49.99, 'stock_
quantity': 200}]
```

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

- **Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).**
- **Implement sorting algorithms to rank stocks by percentage change.**
- **Implement a search function that retrieves stock data instantly when a stock symbol is entered.**
- **Optimize sorting with Heap Sort and searching with Hash Maps.**
- **Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.**

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

#PROMPT

"""An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.fetch or simulate stock price data (StockSymbol,

Opening Price, Closing Price).

• Implement sorting algorithms to rank stocks by percentage change.

• Implement a search function that retrieves stock data instantly when a stock symbol is entered.

• Optimize sorting with Heap Sort and searching with Hash Maps.

• Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs."""

#CODE

```
def main():
    # Simulated stock data
    stocks = [
        {"symbol": "AAPL", "opening_price": 150.00, "closing_price": 155.00},
        {"symbol": "GOOGL", "opening_price": 2800.00, "closing_price": 2750.00},
        {"symbol": "AMZN", "opening_price": 3400.00, "closing_price": 3450.00},
        {"symbol": "MSFT", "opening_price": 300.00, "closing_price": 310.00},
        {"symbol": "TSLA", "opening_price": 700.00, "closing_price": 680.00},
    ]

    # Calculate percentage change for each stock
    for stock in stocks:
```

```
    stock["percentage_change"] = ((stock["closing_price"] - stock["opening_price"]) /
stock["opening_price"]) * 100

    # Sort stocks by percentage change using Heap Sort
    sorted_stocks_heap = heap_sort(stocks)
    print("Stocks sorted by percentage change (Heap Sort):")
    for stock in sorted_stocks_heap:
        print(f"{stock['symbol']}: {stock['percentage_change']:.2f}%")

    # Search for a specific stock symbol using Hash Map
    stock_map = {stock["symbol"]: stock for stock in stocks}
    search_symbol = "AAPL"
    if search_symbol in stock_map:
        print(f"Stock data for {search_symbol}: {stock_map[search_symbol]}")
    else:
        print(f"Stock symbol {search_symbol} not found.")
def heap_sort(arr):
    """Heap Sort is a comparison-based sorting algorithm that uses a binary heap data
structure to sort elements. It first builds a max heap from the input data and then
repeatedly extracts the maximum element from the heap and rebuilds the heap until it is
empty.

    Time Complexity:
    - Average Case: O(n log n)
    - Best Case: O(n log n)
    - Worst Case: O(n log n)

    Args:
    arr (list): The list of elements to be sorted.

    Returns:
```

```
    list: A new sorted list.
    """
    n = len(arr)

    # Build a maxheap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # One by one extract elements from heap
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]  # swap
        heapify(arr, i, 0)

    return arr
def heapify(arr, n, i):
    """To heapify a subtree rooted with node i which is an index in arr[]. n is the size of the heap."""
    largest = i  # Initialize largest as root
    l = 2 * i + 1  # left = 2*i + 1
    r = 2 * i + 2  # right = 2*i + 2

    # If left child is larger than root
    if l < n and arr[l]["percentage_change"] > arr[largest]["percentage_change"]:
        largest = l

    # If right child is larger than largest so far
    if r < n and arr[r]["percentage_change"] > arr[largest]["percentage_change"]:
        largest = r

    # If largest is not root
```

```
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]  # swap


        # Recursively heapify the affected sub-tree
        heapify(arr, n, largest)
if __name__ == "__main__":    main()
```

#OUTPUT

```
Stocks sorted by percentage change (Heap Sort):
TSLA: -2.86%
GOOGL: -1.79%
AMZN: 1.47%
MSFT: 3.33%
AAPL: 3.33%
Stock data for AAPL: {'symbol': 'AAPL', 'opening_price': 150.0, 'closing_price': 155.0, 'percentage_change': 3.3333333333333335}
```